

EX.NO:7 IMPLEMENTATION OF A MULTI-THREAD

PROGRAM:

```
import java.util.*;
class even implements Runnable
{
    public int x;

    public even(int x)
    {
        this.x = x;
    }
    public void run()
    {
        System.out.println("New Thread " + x + " is EVEN and Square of " + x + " is: " + x *
x);
    }
}
class odd implements Runnable
{
    public int x;

    public odd(int x)
    {
        this.x = x;
    }
    public void run()
    {
        System.out.println("New Thread " + x + " is ODD and Cube of " + x + " is: " + x * x
* x);
    }
}
class A extends Thread
{
    public void run()
    {
        int num = 0;
        Random r = new Random();
        try
        {
            for (int i = 0; i < 5; i++)
            {
                num = r.nextInt(100);
                System.out.println("Main Thread and Generated Number is " + num);
                if (num % 2 == 0)
                {
                    Thread t1 = new Thread(new even(num));
                    t1.start();
                }
            }
        }
        else
        {
            {
```

```

        Thread t2 = new Thread(new odd(num));
        t2.start();
    }
    Thread.sleep(1000);
    System.out.println("  ");
}
}
catch (Exception ex)
{
    System.out.println(ex.getMessage());
}
}
}
public class ThreadProgram
{
    public static void main(String[] args)
    {
        A a = new A();
        a.start();
    }
}

```

OUTPUT:

Main Thread and Generated Number is 26
 New Thread 26 is EVEN and Square of 26 is: 676

Main Thread and Generated Number is 15
 New Thread 15 is ODD and Cube of 15 is: 3375

Main Thread and Generated Number is 82
 New Thread 82 is EVEN and Square of 82 is: 6724

Main Thread and Generated Number is 7
 New Thread 7 is ODD and Cube of 7 is: 343

Main Thread and Generated Number is 40
 New Thread 40 is EVEN and Square of 40 is: 1600

EX.NO.8

INTER-THREAD COMMUNICATION

PROGRAM:

```
class SharedResource
{
    private int data;
    private boolean available = false;
    public synchronized void produce(int value)
    {
        while (available)
        {
            try
            {
                wait();
            }
            catch (InterruptedException e)
            {
                System.out.println("Thread interrupted");
            }
        }
        data = value;
        System.out.println("Produced: " + data);
        available = true;
        notify();
    }
    public synchronized void consume()
    {
        while (!available)
        {
            try
            {
                wait();
            }
            catch (InterruptedException e)
            {
                System.out.println("Thread interrupted");
            }
        }
        System.out.println("Consumed: " + data);
        available = false;
        notify();
    }
}

class Producer implements Runnable
{
    private SharedResource resource;

    public Producer(SharedResource resource)
    {
        this.resource = resource;
    }

    public void run()
```

```

    {
        for (int i = 1; i <= 5; i++)
        {
            resource.produce(i);
            try
            {
                Thread.sleep(100);
            }
            catch (InterruptedException e)
            {
                System.out.println("Thread interrupted");
            }
        }
    }
}

class Consumer implements Runnable
{
    private SharedResource resource;

    public Consumer(SharedResource resource)
    {
        this.resource = resource;
    }
    public void run()
    {
        for (int i = 1; i <= 5; i++)
        {
            resource.consume();
            try
            {
                Thread.sleep(100);
            }
            catch (InterruptedException e)
            {
                System.out.println("Thread interrupted");
            }
        }
    }
}

public class InterThreadCommunication
{
    public static void main(String[] args)
    {
        SharedResource resource = new SharedResource();
        Thread producerThread =
            new Thread(new Producer(resource));
        Thread consumerThread =
            new Thread(new Consumer(resource));
        producerThread.start();
        consumerThread.start();
    }
}

```

OUTPUT:

Produced: 1
Consumed: 1
Produced: 2
Consumed: 2
Produced: 3
Consumed: 3
Produced: 4
Consumed: 4
Produced: 5
Consumed: 5